Simulation-based Test Case Generation for Unmanned Aerial Vehicles in the Neighborhood of Real Flights

Sajad Khatiri
Software Institute - USI, Lugano & Zurich University of Applied Sciences

Sebastiano Panichella
Zurich University of Applied Sciences (ZHAW)

Paolo Tonella
Software Institute - USI, Lugano

***Abstract*—Unmanned aerial vehicles (UAVs), also known as drones, are acquiring increasing autonomy. With their commercial adoption, the problem of testing their functional and non-functional, and in particular their safety requirements has become a critical concern. Simulation-based testing represents a fundamental practice, but the testing scenarios considered in software-in-the-loop testing may not be representative of the actual scenarios experienced in the field.**

In this paper, we propose SURREAL (teSting Uavs in the neighboRhood of REAl fLights), a novel search-based approach that analyses logs of real UAV flights and automatically generates simulation-based tests in the neighborhood of such real flights, thereby improving the realism and representativeness of the simulation-based tests. This is done in two steps: first, SURREAL faithfully replicates the given UAV flight in the simulation environment, generating a simulation-based test that mirrors a pre-logged real-world behavior. Then, it smoothly manipulates the replicated flight conditions to discover slightly modified flight scenarios that are challenging or trigger misbehaviors of the UAV under test in simulation. In our experiments, we were able to replicate a real flight accurately in the simulation environment and to expose unstable and potentially unsafe behavior in the neighborhood of a flight, which even led to crashes.

***Index Terms*—Autonomous Systems, Software Testing, Simulation, Flight Replication**

I. Introduction

With the boost of *cyber-physical systems* (CPS) in both academia and industry over the past decade, we have witnessed impressive advancements in the technology available in healthcare, avionics, automotive, railway, and robotics sectors [1], [2]. Unmanned Aerial Vehicles (UAVs) [3] or drones equipped with onboard cameras and sensors have already demonstrated that autonomous flights are possible in real environments. This sparked great interest in a plethora of application scenarios, with crop monitoring [4], surveillance [5], medical and food delivery [6], and search and rescue in disaster areas [7] representing only some of the relevant applications of UAVs.

Support for UAV developers has increased over the years, with open-access projects for the software (i.e., firmware) and hardware (e.g., flight controller). Well-known examples are Ardupilot [8] and PX4 [9] (autopilot software) and Pixhawk [10] (open standards for UAV hardware). On the other hand, automated testing of UAVs (and in general, CPS) to ensure their proper behavior represents still an open research challenge [11], [12], [13], [14]. Simulation-based testing is a promising direction to improve UAV testing practices [15], [16], [17]. Researchers proposed the use of *digital-twins*, i.e., virtual representations of real-time, physical objects or processes, to simulate and test CPS in a diversified set of scenarios [18], [19], [20], [21], [22], [23], and support testing automation [24], [25]. However,it is challenging to capture the same bugs as physical tests in simulation [17], [11] and to generate representative simulation-based test cases that expose realistic bugs [15].

To better illustrate the problem statement, let us consider the following scenario: *Bob* is a UAV customer using a quad-copter based on PX4 [9] (a popular open-source UAV firmware enabling autonomous flight, path planning, and obstacle avoidance) for crop monitoring missions over various croplands. Since some of these lands are close to or include trees, buildings, roads, or other populated areas and facilities, he is particularly concerned about the safety and reliability of his quad-copter during the missions. He has already tested the UAV in a specific scenario over one of these lands: an autonomous flight from a starting point S to a destination point D, crossing a small building on the way. *Bob* observed that the UAV reached the destination safely while avoiding obstacles in the scene, but he is not yet convinced that it will be the case in other possible scenarios and over other lands. Specifically, he is interested to know if the UAV would still complete the mission safely, even if the scenario was a bit different, e.g., the building had a different size, the planned path, or the weather was different.

Since he does not have the budget to test the UAV in all such variations manually in the field, *Bob* contacts *Alice*, an experienced PX4 developer, to help in the (safety) assessment of his UAV in such diversified scenarios. As the first step, *Alice* asks *Bob* for the *Flight Logs* of his field tests, since she knows the logs include very valuable information about how the environment was perceived by the UAV during the flights.

Now, *Alice* has the challenge of manually analyzing the flight logs, interpreting the results of the test, and investigating ways to make a proper assessment of the drone in the alternative, neighboring flight scenarios. As a practical and viable strategy, *Alice* decides to use simulators (e.g., Gazebo[26]) to replicate the real test flights in simulation, and to identify close-related scenarios that could potentially fail in the real

world. However, the problem of replicating the logged real flight in simulation with high fidelity remains a big issue for *Alice*. Here, the questions are: 1) how to enable *Alice* to faithfully replicate a real UAV flight in simulation, by analyzing the flight log coming from an unknown environment? and 2) how to enable *Alice* to test the UAV in a set of diversified possible scenarios in the neighborhood of the given field test?

In this paper, we address such problems by proposing **SURREAL** (te**S**ting **U**avs in the neighbo**R**hood of **REA**l f**L**ights), a novel, search-based approach that analyses real-world UAV flight logs and automatically generates simulation-based tests in the neighborhood of real flights, thereby improving the realism of simulation-based tests.

Software engineering researchers proposed several automated solutions to generate test cases reproducing the crashes of software-only systems [27], [28], [29], [30], [31], [32]. However, to the best of our knowledge, no existing approach for CPS/UAV testing [33], [16] addresses the problem of test scenario replication and test case generation, where the execution state to be reproduced is not only the state of the program, but it involves also the state of the real world.

By using our approach, the work done by *Alice* in the previous scenario is drastically reduced. Indeed, *Alice* can download the flight log provided by *Bob* and give it as input to SURREAL. Following the steps described in Section III, our approach automatically generates simulation-based tests in the neighborhood of the real flight from *Bob*. This is done by first replicating the UAV behavior in the simulation environment, and then smoothly manipulating the replicated flight conditions to discover the flight scenarios that trigger unsafe behavior of the UAV in simulation. SURREAL also helps *Bob* to identify potential corner cases and guide his field testing activity.

This paper provides the following contributions:

- A generic approach for automatically generating a simulation-based test case that replicates a real flight scenario, by searching for optimal simulation environment configurations using only the flight log.
- A generic approach that automatically modifies a (replicated) simulation-based test case to generate more challenging test scenarios.
- An empirical evaluation of a specific instance of the generic approaches, for optimal placement of obstacles in the simulation environment during an autonomous flight.
- A replication package, available on Zenodo [34], including the datasets, experiment results, and (after acceptance) the tool SURREAL.

II. Background

This section provides an overview of the UAV Architecture and UAV Firmware and Software used in our research.

A. UAV Architecture

UAVs are characterized by the Perception, Planning, and Control [35] software components, and the hardware components that interact with the environment and the UAV software.

The **Perception component** is responsible for the UAV's understanding and modeling of the surrounding environment based on sensor signals. The functionalities of this component include *state estimation* [36], [37] and *mapping* [38]. State estimation recreates the drone state in the environment and enables navigation and autonomous movement [37], while mapping strategies compute obstacle distances, to create a model of the surrounding area [38]. The **Planning component** aims at finding an optimal trajectory from starting point to the destination, e.g., by computing *polynomial trajectories* [39], [40] and then applying *trajectory optimization* [41]. The **Control component** determines the actuator control commands to be executed by the UAV to safely navigate the environment and enables the autopilot (onboard commands) and/or the ground-control station (commands from a remote station) modalities [42], [35], [43].

B. UAV Firmware and Software

1) PX4 Platform: PX4 [9] open-source flight control platform is often used to implement a UAV system. PX4 supports Software In-the-Loop (SIL) simulation to safely execute UAV flights in simulation environments, with the purpose of checking novel control algorithms before actually flying the UAV, limiting the risk of damaging the vehicle. It also supports Hardware In-the-Loop (HIL) simulation, by providing simulation inputs to the firmware deployed on a real flight controller board.

2) PX4 Simulation Environments: Simulators allow PX4 to control a modeled vehicle in a *simulated world*. Hence, PX4 communicates with a simulator (e.g., Gazebo [26]) to receive sensor data from the simulated world and send actuator control commands back. In this setting, the UAV pilot (user), similarly to a real vehicle, can interact with the simulated vehicle using a ground control station (GCS), a radio controller (RC) or an offboard API (e.g. ROS), both to send control commands and to receive telemetry data. PX4 supports several HIL and SIL simulators [44]. In the context of our work, we considered Gazebo [26] as PX4's reference 3D simulation environment since it is particularly suitable for testing its obstacle avoidance and computer vision functionalities.

3) Flight Logs: PX4 logs any message communicated between RC or GCS and UAVs, or between its internal modules [45]. This includes the sensor outputs, location, other estimations based on sensor readings, the commands sent to the UAV, and the errors/warnings from the internal modules. Logs are stored on the UAV file system after each flight, to help investigate issues encountered during a flight and their root causes [45]. A sample public flight log hosted on PX4 Flight Review website [46] can be accessed with the link in the footnote[1].

4) Flight Modes: Flight modes define how the autopilot responds to RC input, and how it manages the vehicle movements during fully autonomous flights. Flight modes provide different levels of autopilot assistance, ranging from automation of common tasks (e.g., takeoff and landing) or flying a

[1] logs.px4.io/plot_app?log=fe467648-0041-4279-a3d2-d065b77b6a43

preplanned path, to mechanisms that make it easier to hold a certain altitude level or position when needed. Flight modes can be divided into *manual* and *autonomous* modes. Manual modes allow the user to control the vehicle movement via the RC sticks, while autonomous modes are fully controlled by the autopilot, with no pilot/RC input. During an autonomous flight, obstacle avoidance [47] can be enabled to let the UAV locate any obstacle on its path using its onboard cameras, and navigate around them safely to reach the destination.

III. Approach

The de-facto standard testing process of UAVs relies on *manually-written system-level tests* for testing UAVs *in the field*. These tests are defined as software *configurations* in a given *physical environment* and a set of runtime *commands* that make the UAV fly with a specific observable *behavior* (e.g., flight trajectory, speed, distance to obstacles). We model a UAV simulated test case with the following *test properties*:

- *UAV Configuration: Autopilot parameters* [2] *set at startup, configuration files (e.g., mission plan) required.*
- *Environment Configuration:* Simulation settings such as simulation world (e.g., surface material, UAV's initial position), surrounding objects (e.g., obstacles size, position), weather condition (e.g., wind, lighting).
- *Runtime Commands:* Timestamped external commands sent from GCS or RC to the UAV during the flight (e.g., changing the flight mode, flying in a specific direction, starting autonomous flight).

Since the physical attributes of the *simulated* and *real* UAVs and the surrounding environments are often not identical, simply replaying the same set of commands sent to a physical UAV (as recorded in the logs) would not always result in the same observable behavior in simulators. For instance, sending a command for going forward with full power for 1 second, will likely not bring the real and simulated UAVs to have the same speed and acceleration, and to cover the same distance. This is typically due to the differences in UAV real vs simulated characteristics (e.g., weight, motors power, and sensors accuracy) and to unpredictable environmental factors (e.g., wind and other disturbances).

Given a field test log, SURREAL aims to generate simulated test cases that replicate, as closely as possible, real-world observations (e.g., flight trajectory). This is done by finding the best combination of the above-mentioned test properties, so as to minimize some distance measure between the sensor readings of the field test and its simulated counterpart (e.g., the Euclidean distance between the two, respective flight trajectories).

Starting from this replicated simulation test, SURREAL generates variants in the close neighborhood of the test case, with the goal of creating potentially more challenging scenarios. This is achieved by updating the test properties, with the purpose of increasing the complexity (or risk level) of the generated test cases. The test complexity is measured according to a given *fitness function*, e.g., the minimum distance of the UAV to the obstacles during the flight. To achieve these goals, we propose a generic search-based approach that generates simulation-based test cases that minimize a given *distance measure* (or maximize a given fitness function) by iteratively manipulating the corresponding test properties.

In the following sections, we first describe the generic approach to generate test cases that optimize a given fitness function. Then we instantiate it for flight trajectory replication during autonomous flights and for the generation of new challenging scenarios, by manipulating obstacles in the environment. It is important to note that the generic approach can also be instantiated to replicate other UAV behaviors (e.g., speed, acceleration, outputs to motors), or generate challenging test cases w.r.t other requirements (e.g., UAV stability, mission duration, power consumption), and by manipulating other test properties (e.g., wind, planned waypoints, runtime commands).

A. Generic Approach

1) Context: The proposed generic approach can be used in two different contexts:

A) *[Flight Replication]* Given a real flight log, the goal is to generate a simulation-based test case that replicates the flight w.r.t. specific UAV behavioral properties. The behavioral properties to reproduce can be any logged variable, such as outputs to the actuators (motors thrust), raw inputs coming from sensors, or higher-level variables calculated from them (e.g., UAV position in the 3D space), with the replication accuracy measured by a *distance metric (or similarity measure)*. For instance, by choosing to replicate the 3D space position variables we create a simulated flight with a similar trajectory as that recorded in the real-world log.

B) *[Test Generation]* Given a simulation-based test case, the goal is to generate variants of such test that are more *challenging* w.r.t specific *complexity measures*. The complexity of the test case is calculated based on the risk level of violating (safety) requirements, such as flying too close to obstacles.

We *formulate both problems as a search problem* focused on finding the optimal test properties that maximize a relevant *fitness function*.

2) Search Algorithm: Algorithm 1 details our approach. Overall, the search is an iterative process that finds the best mutations to apply to the current solution at any given step.

The process starts with an initial *seed solution* (test properties). In the context of flight replication, the seed is available directly from the raw data in the original flight log. It includes the logged drone configurations, RC command series, and potential obstacle information that can be extracted from distance sensors. In the case of test generation, the seed consists of an existing simulation-based test case, from which the algorithm generates more challenging variants.

The second input, *fitness_func*, is the function computing the fitness of the solutions. It gets the simulated flight logs as input, and computes a fitness value according to the given goal (see section III-A4). For flight replication, it consists of a distance metric between the original flight sensor values and

[2] https://docs.px4.io/main/en/advanced_config/parameter_reference.html

the simulated ones, as described in detail in section III-B. For test generation, it consists of a risk assessment measure for the given test case, as described in detail in section III-C.

The third input is the *budget* assigned to the search process: the maximum number of test case evaluations performed before returning the final solution found during the search. We measure the search budget as the number of *allowed test case evaluations* since evaluating a candidate test is the most expensive operation performed by the proposed algorithm, as it consists of a full simulation of the UAV behavior. The final input, *min_rounds* corresponds to the *minimum ensured rounds of global search* (the while loop in lines 6-11).

The initial best solution is set at line 2 as the seed. At line 3, we initialize an empty dictionary, named *evaluation_hash* which will record all the evaluated solutions and their fitness values as the algorithm proceeds. *evaluation_hash* will be used by the function *EVALUATE* (described in section III-A3, pseudo-code not shown for space reasons) to implement *memoization*, i.e., to skip the simulations if the same test properties have been already evaluated in previous iterations and directly set the fitness value as obtained from the dictionary. The *EVALUATE* function is also in charge of updating the execution budget, which is decreased by one when a simulation is needed, while it is left unchanged if the fitness for the requested execution is available from *evaluation_hash* (the last parameter, which is NULL at line 4, is a local budget updated similarly to the global one).

After evaluating the initial seed solution, which consumes one simulation from the budget, the main optimization loop is entered at line 6. The loop terminates when the local search, invoked at line 10, is unable to find a better solution for all parameters that can be mutated in the test properties. We assume here that test properties come with *mutators*, i.e., parameterized operators that can be applied individually to each test property. For example, the property $obstacle.position.x$ can be modified by an additive mutator, which moves the current position of the obstacle in the simulation environment along the x axis by a parameter value called $MOVE_X$.

The *for* loop at line 9 iterates over all mutators available for the given test properties and tries to optimize each of them individually by invoking a local optimization procedure at line 10. The local budget is obtained by uniformly distributing the available budget across all mutators in the remaining ensured rounds (line 8), with each local search not necessarily consuming entirely its local budget. Hence, when re-entering the while loop at line 6, the residual global budget might still be available for the next iteration.

Algorithm 2 shows the details of the local search. This algorithm can be classified as an *adaptive greedy algorithm* that searches the parameter space of each mutator. We chose a greedy technique to reduce the number of simulations needed to reach the optimum (executing an entire simulation is computationally expensive). At the same time, to avoid the choice of a sub-optimal greedy optimization step, we adapt the *step* parameter as the algorithm progresses.

Each mutator comes with a default value and a default step.

Algorithm 1: GENERIC-TEST-PROPERTIES-SEARCH

```
Input: seed: original test properties
       _func: fitness function to maximize
       budget: global search budget (max num of simulations)
       min_rounds: minimum ensured round of global search
Result: best: test properties that optimize the fitness
1  begin
2      best = seed
3      evaluation_hash = {}
4      EVALUATE(fitness_func, best, evaluation_hash, budget,
         NULL)
5      improved = true
6      while improved do
7          improved = false
8          local_budget = budget/(|seed.mutators|×min_rounds)
9          for mutator in seed.mutators do
10             improved = improved ∨
                 LOCAL-TEST-PROPERTIES-SEARCH(best,
                 mutator, evaluation_hash, local_budget, budget)
11         min_rounds = min_rounds - 1
12     return best
```

The default value is a value that leaves the test properties unchanged. For a multiplicative mutator, it is 1; for an additive mutator, 0. The default step is mutator specific. For example, the mutator that moves the obstacle along the x axis has a default value of 0, because it is additive, and has a default step of 4 meters. The default step is defined specifically for each mutator parameter, based on the expected parameter range. Its value is not critical, as the local search adjusts it in an adaptive way. At lines 2-3 the default step and parameter value for the given mutator are assigned to the variables *step*, *param*.

The optimization loop starts at line 7, with 2 termination conditions: the local budget has expired, or there is no improvement of the best solution for more than *MAX_IT* iterations.

In lines 8-9 we create two mutated solutions by either increasing or decreasing the mutator parameter by the current optimization step. These candidate solutions are evaluated at lines 10-11 (function *EVALUATE* will skip simulation if the test properties can be found in *evaluation_hash*). Then, if either the first mutated solution or the second one improves the current best solution by a margin higher than ϵ, the new best solution is recorded. Otherwise, if we are in a plateau (line 29) the optimization loop stops, while if both mutations *mu1, and mu2* have decreased the fitness value by an amount greater than ϵ (*else* case at line 28 with a *false* condition at line 29), the optimization step is halved adaptively (line 31).

If the same step is applied multiple times in the positive (resp. negative) direction, at line 19 (resp. 27) the optimization step is doubled, to converge more quickly to the final solution. The local search terminates by assigning the new best solution to the input/output variable *best*, which is eventually returned by Algorithm 1. It returns *true* if a better solution was found during the local search; *false* otherwise.

Algorithm 2: LOCAL-TEST-PROPERTIES-SEARCH

```
   InOut: best: best solution found so far
   Input : mutator: test property mutator
   InOut: evaluation_hash: memory of past evaluations
   Input : local_budget: max simulations for current mutator
   InOut: budget: overall max simulations allowed
   Result: improved: previous best solution was improved
1  begin
2      step = mutator.default_step
3      param = mutator.default_value
4      positive_moves = negative_moves = 0
5      iter_with_no_improvements = 0
6      new_best = best
7      while local_budget > 0 ∧ iter_with_no_improvements <
        MAX_IT do
8          mu1 = MUTATE(best, mutator, param + step)
9          mu2 = MUTATE(best, mutator, param - step)
10         EVALUATE(mu1, evaluation_hash, budget,
            local_budget)
11         EVALUATE(mu2, evaluation_hash, budget,
            local_budget)
12         if mu1.fitness > new_best.fitness + ϵ ∧ mu1.fitness >
            mu2.fitneess then
13             new_best = mu1
14             param = param + step
15             positive_moves += 1
16             negative_moves = 0
17             iter_with_no_improvements = 0
18             if positive_moves > MAX_SEQ_IT then
19                 step = step · 2
20         else if mu2.fitness > new_best.fitness + ϵ then
21             new_best = mu2
22             param = param - step
23             negative_moves += 1
24             positive_moves = 0
25             iter_with_no_improvements = 0
26             if negative_moves > MAX_SEQ_IT then
27                 step = step · 2
28         else
29             if |new_best.fitness - mu1.fitness| < ϵ ∧
                |new_best.fitness - mu2.fitness| < ϵ then
30                 break
31             step = step / 2
32             positive_moves = negative_moves = 0
33             iter_with_no_improvements += 1
34     improved = false
35     if new_best ≠ best then
36         best = new_best
37         improved = true
38     return improved
```

3) Solution Evaluation: To evaluate a search solution, i.e., the candidate test properties, we generate and execute the corresponding simulated test case automatically. The test case automates all necessary steps: setting up the test environment, building and running the firmware code, configuring the simulator with the simulated world properties, connecting the simulated UAV to the firmware, and applying the UAV configurations from the test case properties at startup. Then, the test case commands are scheduled and sent to the UAV, the flight is monitored for any issues, and after test completion, the flight log file is extracted. Due to the nature of the control mechanisms and the surrounding environment, the UAV behavior (both in simulation and in the real world) can be non-deterministic. To eliminate the effects of outliers in our experiments, we run each test case *n* times, extract the logs, and use the average of the variables recorded in the logs for computing the fitness function.

4) Fitness Function: Our search algorithm has the overall goal of maximizing the given fitness function (*fitness_func*) provided as input to the algorithm with the following signature:

$fitness_func(flight_logs : List < Log >) \rightarrow float$

The input is a list of flight logs, obtained from multiple executions of the same simulation-based test case. The output is a numeric value measuring the overall fitness of the test case w.r.t. our goal. Since the fitness function is maximized by the local search algorithm (see lines 12, 20), when the goal is to minimize some metric value (e.g., the distance d between trajectories) we supply the negation of the metric value ($-d$) as the fitness function.

B. Flight Replication in Autonomous Mode

1) Context: Given the flight log and the mission configuration of an autonomous flight that includes a given number of N ($\geqslant 1$) obstacles, with unknown size and position, we generate a simulated test case that includes the optimized size and position of the obstacles, with similar UAV trajectory in simulation. We propose an instance of our generic flight replication algorithm for this problem. In PX4's autonomous mode, the mission is uploaded to the UAV in advance. After the mission *start* command, the UAV follows the mission waypoints in a completely autonomous way. If obstacle avoidance [47] is enabled, the UAV will use its distance sensors and camera to locate any obstacle on the way and will automatically find its way to the next waypoint beyond the obstacle.

2) Fitness Function: Since the relevant logged variables are time series, the fitness function must be able to compare two time series, measuring the distance between the sequence of replicated states from the *simulation log* (intermediate solutions of the search algorithm) and the sequence of expected states from the *original log* (real-world flight to be replicated in simulation). As a general-purpose fitness function that could potentially work on any metrics from the original log, we use Dynamic Time Warping (DTW) [48], a well-known distance measure for multi-dimensional time series of different lengths that has already been used for comparing UAV flight trajectories [49]. DTW is based on a dynamic programming algorithm that matches the elements appearing in two sequences s, t by finding the pairing that minimizes the overall cost. The minimum cost for pairing the element in position i from s with the element in position j from t is recursively determined as:

$$DTW[i,j] = d(s_i, t_j) + \min\{DTW[i-1,j], DTW[i,j-1], DTW[i-1,j-1]\} \quad (1)$$

where $d(s_i, t_j)$ is the distance between the metric values appearing at position i (resp. j) in the two logs (in the simplest case, just the Euclidean distance $\|s_i - t_j\|$), while the recursive choice of the minimum DTW value corresponds respectively to advancing the pairwise comparison by one position along the first list only, the second only, or both. Here, the compared sequences s and t are UAV trajectory points $\langle x, y, z \rangle$.

Since DTW compares two time series, not two sets of time series obtained from multiple runs of the same test scenarios, we also need an *aggregation function* to group the logs obtained from multiple runs of the same test scenario into one, coherent time series. For this, we adopt *DTW Barycenter Averaging* [50], an averaging method for time series data that is able to determine and keep the shape of the series, while computing the average.

3) Test Case Properties: The physical world is assumed to be a plain area with N obstacles of predefined shape (e.g., box). Since the flight is operated in autonomous mode, the only variable properties of the generated test cases are the obstacle properties, i.e., the size $(length, width, height)$, position (x, y, z), and rotation angle (r) of each obstacle.

4) Mutation Operators: To search for the optimal obstacle properties, we use the following mutation operators for any of the N obstacles individually:

a) Obstacle Move: The Obstacle Move mutation operator moves the obstacle from the previous location in the simulated world by Δ_x, Δ_y parameters. The default values of these parameters (Algorithm 2, line 3) are 0, while the default step (Algorithm 2, line 2) is 4m. We ignore the z dimension since we assume the boxes are always placed on the ground.

b) Obstacle Resize: This mutation operator resizes the obstacle in place (keeping the center position) by the values provided as $(\Delta_l, \Delta_w, \Delta_h)$. The default value of these parameters (Algorithm 2, line 3) are 0, while the default step (Algorithm 2, line 2) is 4m.

c) Obstacle Rotate: This mutation operator rotates the obstacle around its geometric center in the x,y plane by the mutation operator's parameter Δ_r degrees. The default value of this parameter (Algorithm 2, line 3) is 0, while the default step (Algorithm 2, line 2) is 30 degrees.

5) Seed: If no information on the placement and size of the N obstacles is available in the original log, we create an initial seed solution by randomly placing N obstacles in positions that intersect with the mission flying area.

C. Test Case Generation in Autonomous Mode

To find challenging and buggy UAV fly conditions, we designed another instance of the generic approach (Algorithms 1, 2), which generates challenging test scenarios, starting from the output of flight replication. The search seeds are the final solutions of the algorithms instantiated in Section III-B.

1) Context: Given a simulated test case configuration for autonomous flight (the mission waypoints and obstacle locations and sizes), we want to generate a more challenging simulated test case by introducing an additional obstacle, to force the UAV to get too close to the obstacle (i.e., having a distance below a predefined safety threshold) while still completing the mission. This will create a risky environment for the UAV to operate the mission.

Most of the algorithm is identical to the algorithm instance described in Section III-B for autonomous flight replication, with few modifications. Specifically, we use the same test case properties (location, size, and rotation of the additional obstacle) and the same mutation operators (move, resize and rotate). The fundamental distinction is in the fitness function.

2) Fitness Function: We define the fitness such that the algorithm is guided to get the drone close to all the obstacles in general and close to the border of one obstacle in particular. Correspondingly, the fitness function has two components:

$$\begin{aligned} \textit{sum_dist} &= \min_{p \in \text{trj.points}} \sum_{o \in \text{obs}} d(p, o) \\ \textit{min_dist} &= \min_{o \in \text{obs}, p \in \text{trj.points}} d(p, o) \\ \textit{fitness} &= \textit{sum_dist} + 2 \times \textit{min_dist} \end{aligned} \tag{2}$$

sum_dist accounts for the minimum distance of a single trajectory point to all of the obstacles combined (favoring obstacles closer to each other), while *min_dist* accounts for the minimum distance of the trajectory to any of the obstacles (to be compared against the safety distance). We give the *min_dist* component a weight that is twice the weight of the *sum_dist* component, because it is expected to contribute the most to the generation of risky test scenarios.

In this case, since in multiple runs of the same test case (parallel simulations) the flight trajectories can differ significantly from each other in corner cases (test cases with low *min_dist*), instead of taking the average trajectory, we take the trajectory with the lowest fitness value when returning the fitness value for a given mutation of a test scenario.

IV. Experimental Results

The *goal* of our empirical evaluation is to assess SURREAL's ability to replicate a logged UAV flight and to manipulate it to expose challenging flight conditions. In this section, we elaborate on the research questions, evaluation scenarios, and the results obtained when evaluating our approach.

A. Research Questions

a) ***RQ***$_1$ ***[flight replication]****: Can we generate simulated test cases that faithfully replicate autonomous flight trajectories?* The goal is to replicate the test environment in a way that makes the simulated flight trajectory as similar as possible to the original one, using only logged data as input information. The variable test properties are the environment configurations where the UAV flies a predefined mission autonomously. We investigate an environment setup where placing an obstacle on the map can influence the trajectory, making it more or less similar to the logged one.

b) ***RQ$_2$ [test generation]****: Can we modify simulated test case properties of autonomous flights to make them more challenging for the UAV autonomous controller?* The goal is to investigate the possibility of generating more challenging test cases based on an existing one, replicated from a real-world test. Specifically, we investigate a similar environment setup as RQ$_1$ where placing an additional obstacle properly can result in unsafe or faulty behavior of the UAV.

B. Subject and Original Flights

The *subject* of our experiments is PX4's [9] Autopilot controlling a quad-copter. For RQ$_1$, we consider an original flight conducted in a real-world environment containing an obstacle along the mission route. We set up the *PX4 Vision Autonomy Development Kit*[3], enable module PX4-Avoidance [47], and define a survey mission consisting of taking off to 3 meters altitude, flying towards a waypoint at about 20m distance, and landing. The test field is an empty parking area, with a cargo container sized about $2.5m \times 12m \times 2.5m$ placed in the middle. The original flight trajectory, as extracted from the flight log, is shown in Figure 1 in both plots on the left as a blue line.

For RQ$_2$, we consider a simulated test case in Gazebo [51] with a similar setup (taking off to 10 meters altitude, flying towards a waypoint at about 50m distance, and flying back to the landing point, about 12m to the left of the starting point). We put a box-like obstacle (representing a small building) sized $8m \times 5m \times 20m$ on the direct route towards the destination. Then, we ran SURREAL to generate more challenging environment configurations, obtained by adding a second obstacle to the environment. The flight trajectory, as extracted from the flight log, is shown in Figure 2 (middle).

C. Metrics and Experimental Procedure

To run our experiments, we set the hyper-parameters of Algorithms 1,2 (see Section III-A) to the values reported in Table I. To evaluate our approach, we run SURREAL to replicate the flight trajectory (RQ$_1$) or generate test cases (RQ$_2$), applying 10 repetitions with the same configurations, to gain statistical validity of our results. To deal with the non-determinism of simulated trajectories, at each step of the algorithms, we run multiple simulations in parallel (5 for RQ$_1$ and 10 for RQ$_2$ because of the higher non-determinism in the corner test cases; see parameter *sim. runs* in Table I). We run SURREAL inside Docker containers in a Kubernetes cluster, with main algorithm containers limited to 1.5 virtual CPUs (VCPUs) and 15GB of Ram and simulation containers (running PX4 and Gazebo) limited to 6 VCPUs and 2 GB.

For the best found solutions of each algorithm repetition, we computed the metrics in Table II. We measure the reduction in DTW distance from the original (*org*) when flight reproduction is achieved by SURREAL (*best*) with respect to the search *seed*, as well as the reduction in Fréchet distance [52] for RQ$_1$. Fréchet distance is defined as the maximum distance observed when traveling through the two trajectories (original and replicated flights), considering an optimal mapping of the points visited along the two trajectories. For RQ$_2$, the seed for the additional obstacle has the same size as the first obstacle and is placed $15m$ to the right side of it, so that it does not affect the flight trajectory compared to the original test (figure 2, middle). To ensure the realism of the second obstacle, we kept its initial size and angle and used only the *Obstacle Move* mutation. For RQ$_2$, we measure the reduction of the minimum distance between the UAV and obstacles as well as the percentage of failing (crashing to obstacle) and unsafe (getting closer than 1.5m) simulations for the best generated test cases. We also report the needed evaluation budget (solution evaluations through simulation) to reach the best solutions and the average evaluation time for each solution (run the parallel simulations and process the logs). For each of these metrics, we report the average across 10 repetitions.

[3]https://docs.px4.io/v1.12/en/complete_vehicles/px4_vision_kit.html

TABLE I: Experiment Hyper-parameters

RQ	Parameter	Value
RQ$_{1,2}$	repetition	10
RQ$_{1,2}$	MAX_SEQ.	5
RQ$_{1,2}$	MAX_IT	5
RQ$_{1,2}$	default_step	4m (MOVE, RESIZE), 30° (ROTATE)
RQ$_{1,2}$	ϵ	0
RQ$_1$	budget	200 (seed 1), 100 (seed 2)
RQ$_1$	min_rounds	4 (seed 1), 2 (seed 2)
RQ$_1$	sim. runs	5
RQ$_2$	budget	50
RQ$_2$	min_rounds	2
RQ$_2$	sim. runs	10

TABLE II: Experiment Evaluation Metrics

Metric	Definition
DTW Reduction (RQ$_1$)	$= 1 - \frac{DTW(best,org)}{DTW(seed,org)}(\%)$
Fréchet Reduction (RQ$_1$)	$= 1 - \frac{Frchet(best,org)}{Frchet(seed,org)}(\%)$
Min_dist Red. (RQ$_2$)	$= 1 - \frac{Min_dist(best)}{Min_dist(seed)}(\%)$
Crash & Unsafe Rate (RQ$_2$)	= % of crash & unsafe in the simulations of best
P-value	comparing seed and best fitness distributions
Effect Size	comparing seed and best fitness distributions
Needed Budget	= # of evaluations to reach best
Eval. Time	= average time for each solution evaluation

Once we have collected all the data, we used statistical tests to verify whether there is a statistically significant difference between the seed and the best solution for both RQs across the algorithm repetitions. We employed parametric tests since the Shapiro-Wilk test revealed that the distributions across all experiments follow a Gaussian distribution ($p \gg 0.05$). Hence, we used the one-way Anova test with a p-value threshold of 0.05. We also computed the effect-size of the observed differences using the Vargha-Delaney ($\hat{A}_{12}$) statistic [53]. The Vargha-Delaney ($\hat{A}_{12}$) statistic classifies the quantitative effect size values into four qualitative levels (*negligible*, *small*, *medium*, and *large*), which are easier to interpret.

D. Results

1) ***RQ$_1$ [flight replication]****:* The seed obstacle is a small ($3m \times 3m \times 3m$) box, aligned with the direct path between takeoff and landing position as extracted from the original log.

We do the experiments with two different positioning of this obstacle as seed, illustrated in Figure 1 (left). One placed right below the center of the direct path (seed 1), so that the UAV is probable to fly around the obstacle from the right side, and the other placed on the opposite side (seed 2), so that the UAV is more likely to fly to the left side. The two choices aim to analyze if the algorithm is equally effective when the starting trajectory is on different sides of the obstacle.

As can be seen in Figure 1, showing the best final solution across 10 runs for Seed 1, SURREAL was able to position and size the obstacle very well, so that the trajectory of the replayed flight is almost identical to the original one (with less than 75cm Fréchet distance). During this specific run, the obstacle was moved 2m upwards, rotated 30° clockwise, and the height was increased by 2.8m by the algorithm over the iterations. Although the final obstacle properties are not identical over the 10 runs, the very low DTW between the simulated and original trajectories shows that we do not need to replicate the exact same obstacle configurations to be able to test the UAV in the same manner.

As reported in Table III, the algorithm was able to locate and size the obstacles consistently well in all 10 runs for both seeds, finding solutions with an average DTW of 63 and 59 respectively, which corresponds to an almost identical trajectory, while reducing the seed DTW by 83% and 48% with respect to the distance obtained with the seed obstacles. The two compared DTW distributions are Gaussian ($p > 0.3$ in the Shapiro-Wilk test), so we could use the Anova test: a low p-value ($\ll 0.01$) and a large effect size ($7.9 \gg 0.8$) suggest that the improvement achieved by the algorithm is statistically significant and extremely large.

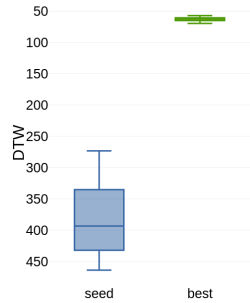
Table III reports both the average required budget (i.e., the average number of evaluations) and the evaluation time (in seconds). With the search budget set to 200 simulations for Seed 1 (100 for Seed 2), on average the final solution was found after 75 (65) evaluations and about 3.5 minutes were used to run all the computations (parallel simulations and distance calculations) at each solution evaluation, adding up to almost 5 hours for each run of the experiment. Figure 2 (left) shows the convergence of the fitness over the iterations. The initial round of mutations (iterations 1-58) are contributing to most improvements in the fitness function, while during the second and third rounds (59-117, 118-165) only marginal improvements are observed.

Discussion on minimum feasible distance: As can be seen in the flight trajectories in Figure 1 (right), the individual runs of the exact same test case can have slightly different trajectories due to the simulation randomness. Although we already mitigated this with the averaging method discussed in Section III-B2, it also limits the minimum DTW distance that can be possibly reached from the original flight. To estimate the potential lower bound due to the simulation randomness, we took the average flight trajectories from the seed solutions of all the 10 runs, and computed their pairwise DTW distance to each other. These distances were in the range $[15.9 - 121.3]$ with an average of 58.5. This means that, due

TABLE III: RQ_1 Evaluation metrics for both seeds

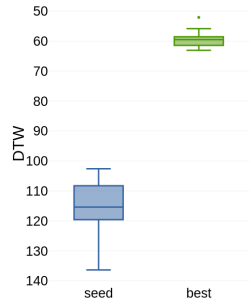
seed 1

Metric	Ave.
Seed DTW	383.5
Best DTW	63.15
DTW Red.	83.1%
Seed Fréchet	6.87 (m)
Best Fréchet	1.14 (m)
Fréchet Red.	83.4 %
P-value	1.9 e-12
Effect Size	7.9
Needed Budget	74.8
Eval. Time	152.9 (s)



seed 2

Metric	Ave.
Seed DTW	116.1
Best DTW	59.1
DTW Red.	48.8%
Seed Fréchet	2.25 (m)
Best Fréchet	1.05 (m)
Fréchet Red.	51.6 %
P-value	1.2 e-12
Effect Size	8.13
Needed Budget	65
Eval. Time	158.9 (s)



to the simulation randomness, even replicating a simulated flight in the same simulator, by putting the exact same test configurations, could reach a DTW distance within this range. Hence, this average (58.5) can be considered as the bottom-line for our optimization process, which indeed reached an average DTW distance of 63.15 when starting from Seed 1; 59.1 when starting from Seed 2.

RQ_1: The information available in the flight logs allows searching for optimal test properties that faithfully replicate UAV *autonomous* flight trajectories in simulation.

*2) **RQ_2 [test generation]:*** As can be seen from the best final solution across 10 runs in Figure 2 (right), SURREAL was able to position and size the second obstacle in a way that the UAV (i) was forced to behave in a non-deterministic way across multiple parallel simulations; (ii) experienced an unsafe behavior, often very close to the first obstacle; (iii) even worse, occasionally crashed into the obstacle, in some simulations. The second obstacle was moved by SURREAL to almost $8m$ to the left and $1.1m$ upwards, making it increasingly harder for the UAV to follow the path. Interestingly, if we position the obstacles closer to each other, the UAV would always act in a more deterministic way, taking a route around the left of the first obstacle, without getting involved in risky situations.

As reported in Table IV, the algorithm was able to find crashing test cases consistently in all 10 runs, forcing the UAV to get as close as $0m$ to the obstacle, down from the $3.3m$ safe distance for the seed. Also, on average, the UAV crashed into the obstacle in around 3 out of the 10 simulations for the best test cases found, and got unsafely close (<1.5m) in 5 more.

The two compared distance distributions are Gaussian ($p >$

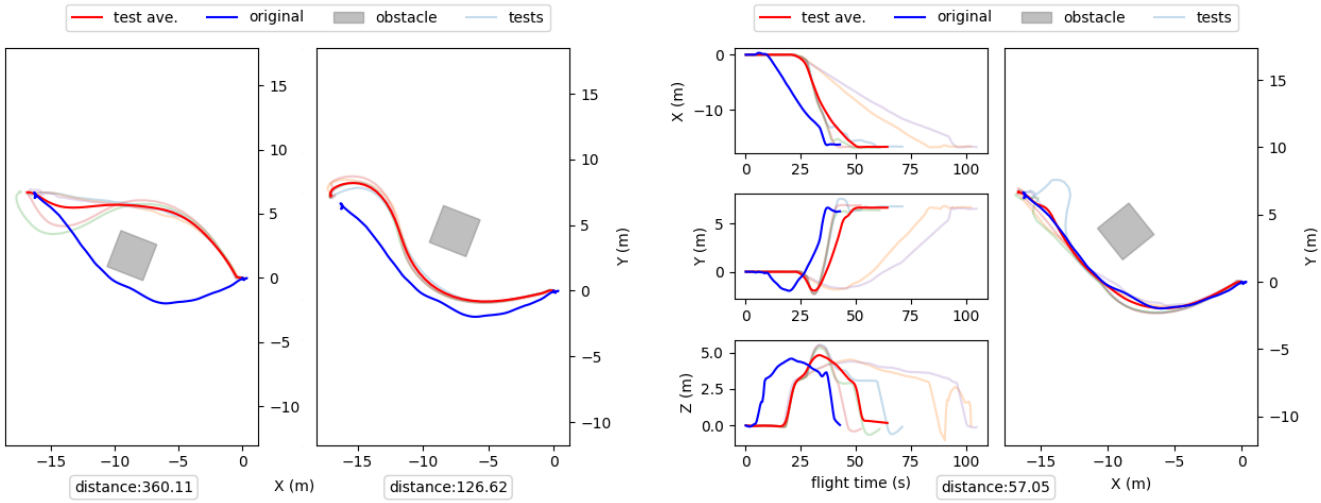
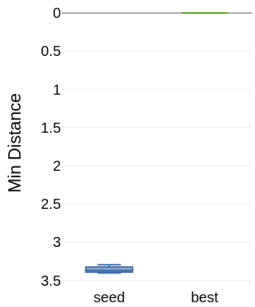


Fig. 1: RQ_1 Seeds 1 and 2 (left) and Final (right) solutions (simulated in Gazebo) compared to the real-world flight

TABLE IV: RQ_2 Evaluation metrics

Metric	Ave.
Seed Min_dist	3.36
Best Min_dist	0
Min_dist Red.	100%
Crash Rate	25 %
Unsafe Rate	84 %
P-value	6.7 e-12
Effect Size	119.9
Needed Budget	36.8
Eval. Time	388.1 (s)



0.69 in the Shapiro-Wilk test), so we could use the Anova test: a low p-value ($\ll 0.01$) and a large effect size ($119.9 \gg 0.8$) suggests that the improvement achieved by the algorithm is statistically significant and extremely large.

RQ_2: Modifying a simulation-based test case allows generating challenging test cases that can expose the UAV to unsafe behaviors or even crashes.

E. Threats to validity

Threats to *construct validity* concern the metrics used to draw a relation between theory and observation. The distance between the trajectory reproduced in the simulator and the original log's trajectory is affected by randomness, due to various sources of noise and non-determinism that affect the simulation environment (e.g., the effect of the wind or the multi-threaded execution in the simulator). Hence, we cannot consider one solution to be closer to the log than another if their trajectories have a small difference (see "Discussion on minimum feasible distance" in Section IV-A). To address this threat we introduced a distance threshold *MIN_DIST* and two trajectories are regarded as equivalent if their distance is lower than *MIN_DIST*. When comparing a simulated trajectory to the log data, we take the average trajectory over 5 simulations to reduce the effects of non-determinism. To gather statistically significant results, we repeated our experiments 10 times. For what concerns the choice of the distance metric used to compare trajectories during the evaluation, we adopted DTW [48] and Fréchet[52], well-known metrics that have already been used before for comparing UAV flight trajectories [49].

Threats to *internal validity* concern the technologies used to generate the UAV scenarios and tests. To increase the generality of our results we could have used also other supported simulators (e.g., jMAVSim). However, it is acceptable to use Gazebo as PX4's reference simulation environment since it is suitable for testing the obstacle avoidance functionalities. Another threat that affects the internal validity is the choice of the seeds for the obstacles used to answer RQ_1 and RQ_2. We used two different seeds for RQ_1 with the flight trajectory being on different sides of them, and a seed additional obstacle for RQ_2 that does not affect the flight trajectory. While our choices were aimed to minimize the effect of the seed solution on the evaluations, a replication with other obstacle types, seed position/size are needed to corroborate our findings.

Threats to *external validity* concern the generalization of our findings. Although we experimented with a widely used UAV firmware and simulator (PX4 and Gazebo), we cannot claim that our results can be generalized to other UAV platforms or other CPS domains. Therefore, further replications and studies considering more CPS domains are desirable.

V. Related Works

Wang *et al.* [17] studied UAV software bugs from UAV Autopilot platforms (PX4 [9] and Ardupilot [8]) and created a taxonomy of UAV bugs and identified their root causes. They report that developers mainly use simulators for reproducing bugs, but setting up realistic-enough simulation environments that capture the same bugs as physical tests is a hard and expensive task. Afzal *et al.* [11], [49] studied the challenges

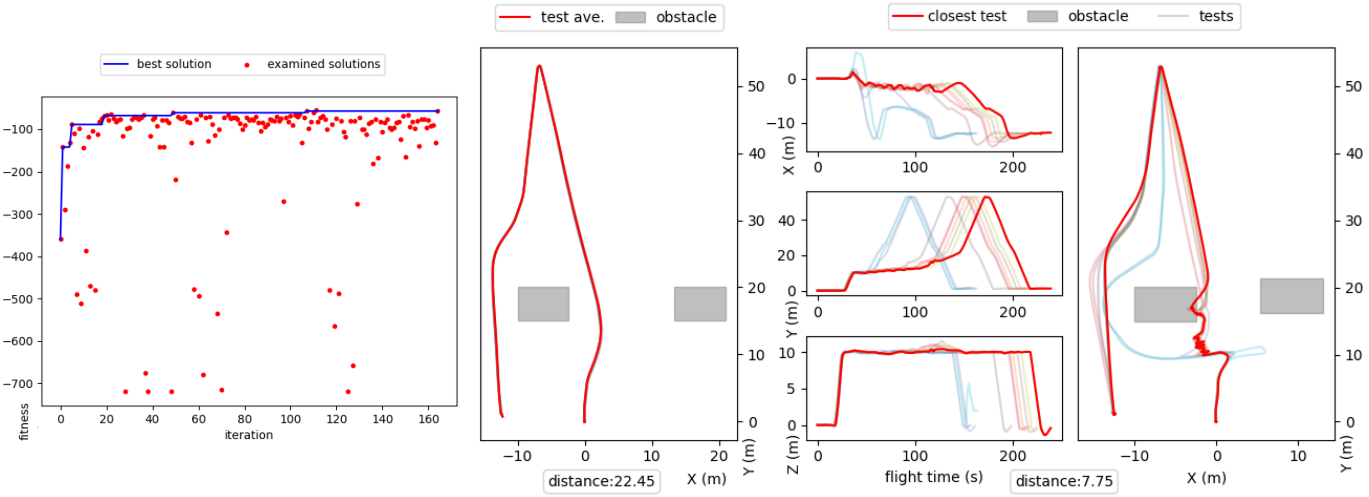


Fig. 2: RQ_1 Fitness progress over iterations (left), RQ_2 Seed (middle) and Final solutions (right)

of testing robotic systems and recognize the *engineering complexity* of the test environment, including the design of realistic inputs to test the system, as one of the biggest challenges. Afzal *et al.* [15] surveyed robotic practitioners on their use of simulators for testing robots and identified the *reality gap* of the robot behavior in simulation and the *reproducibility* of issues encountered in real or simulated tests in simulation as the top challenges they face. Timperly *et al.* [16] conducted an empirical study on fixed bugs in Ardupilot [8] and found that many bugs can be potentially detected before field tests if proper simulation-based testing is in place.

Lindval *et al.* [54] developed a framework for automated testing of autonomous drones in simulation with the aim of solving the test oracle definition problem. Recently, Woodlief et al. proposed PHYS-FUZZ [55], a fuzzing approach tailored specifically for testing mobile robots, taking into account the physical attributes and hazards of such robots. To address the simulation-reality gap, Hildebrandt *et al.* [56] propose a mixed-reality approach for testing UAVs. Their approach, called *world-in-the-loop* simulation, integrates and mixes sensor data from both the simulated and real world, and feeds these mixed sensor inputs to the system under test (UAV).

Complementary, we address UAV simulation-based testing challenges concerning realistic test cases, engineering complexity, and field test reproducibility, by an approach that faithfully replicates real-world test scenarios in simulation and generates similar but challenging variant of these test cases.

Testing of Deep Learning (DL) based systems is a research area that has attracted a growing interest in the last few years. Traditional testing techniques have been adapted to the specific features of machine learning components, addressing problems such as test input generation [57], [58], [59], [60], [61], test oracle definition [62], [63], and test adequacy [64], [57], [65].

In the existing DL testing literature, most related works deal with automated test data generation. Only a few input generators are model-based and expose failures within a simulated environment. Gambi et al. [66], Stocco et al. [62]. Riccio et al. [67], Birchler et al. [21], and Abdessalem et al. [68] test self-driving cars by generating failure-inducing driving scenarios. We share with them the usage of a simulator to control the environment in which an autonomous vehicle is tested, but differently from these works, we aim at generating realistic simulated test cases by first reproducing flying conditions experienced in the field and then manipulating such conditions to identify failure scenarios.

VI. Conclusion and Future Work

Simulation-based testing of UAVs is an important quality assurance step before systems can be released to production. We have proposed a generic adaptive greedy algorithm that can be instantiated to replicate a flight trajectory observed in the field and manipulate it in order to expose misbehaviors. Our experimental results show that SURREAL, implementing our approach, can achieve faithful flight replication by reconstructing the obstacles encountered along the mission's path. After replication, SURREAL is also able to manipulate the obstacles in the environment to find challenging conditions that lead to unsafe behavior of the UAV.

In our future work, we plan to investigate surrogate models that can predict the behavior of the UAV without actually running any simulation. Such models can guide our adaptive greedy search algorithm at low computational cost, making the search more efficient and potentially capable of exploring more complex critical scenarios. We also intend to experiment with additional UAV models and environment configurations, including e.g. different weather conditions and obstacle types.

Acknowledgment

We gratefully acknowledge the Horizon 2020 (EU Commission) support for the project *COSMOS* (DevOps for Complex Cyber-physical Systems), Project No. 957254-COSMOS.

References

[1] C. S. Wickramasinghe, D. L. Marino, K. Amarasinghe, and M. Manic, "Generalization of deep learning for cyber-physical system security: A survey," in *44th Annual Conference of the IEEE Industrial Electronics Society (IECON)*, 2018, pp. 745–751.

[2] H. Chen, "Applications of cyber-physical system: A literature review," *Journal of Industrial Integration and Management*, vol. 02, no. 03, p. 1750012, 2017.

[3] X. Zhang, Y. Liu, Y. Zhang, X. Guan, D. Delahaye, and L. Tang, "Safety assessment and risk estimation for unmanned aerial vehicles operating in national airspace system," *Journal of Advanced Transportation*, 2018.

[4] C. Carbone, D. Albani, F. Magistri, D. Ognibene, C. Stachniss, G. Kootstra, D. Nardi, and V. Trianni, "Monitoring and mapping of crop fields with UAV swarms based on information gain," in *Distributed Autonomous Robotic Systems - 15th International Symposium, DARS 2021, June 1-4, 2021, Online Event*, ser. Springer Proceedings in Advanced Robotics, F. Matsuno, S. Azuma, and M. Yamamoto, Eds., vol. 22. Springer, 2021, pp. 306–319. [Online]. Available: https://doi.org/10.1007/978-3-030-92790-5_24

[5] E. Balestrieri, P. Daponte, L. De Vito, F. Picariello, and I. Tudosa, "Sensors and measurements for UAV safety: An overview," *Sensors*, vol. 21, no. 24, p. 8253, 2021. [Online]. Available: https://doi.org/10.3390/s21248253

[6] R. D'Andrea, "Guest editorial can drones deliver?" *IEEE Trans Autom. Sci. Eng.*, vol. 11, no. 3, pp. 647–648, 2014. [Online]. Available: https://doi.org/10.1109/TASE.2014.2326952

[7] S. Chowdhury, O. Shahvari, M. Marufuzzaman, X. Li, and L. Bian, "Drone routing and optimization for post-disaster inspection," *Comput. Ind. Eng.*, vol. 159, p. 107495, 2021.

[8] Ardupilot.org, "Ardupilot – versatile, trusted, open," 2007, accessed: 07.02.2022. [Online]. Available: https://ardupilot.org/

[9] L. Meier, D. Honegger, and M. Pollefeys, "Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2015, pp. 6235–6240.

[10] Pixhawk.org, "Pixhawk | the hardware standard for open-source autopilots." 2021. [Online]. Available: https://pixhawk.org/

[11] A. Afzal, C. Le Goues, M. Hilton, and C. S. Timperley, "A study on challenges of testing robotic systems," in *International Conference on Software Testing, Validation and Verification*. IEEE, 2020, pp. 96–107.

[12] R. Li, H. Liu, G. Lou, J. X. Zheng, X. Liu, and T. Y. Chen, "Metamorphic testing on multi-module UAV systems," in *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*. IEEE, 2021, pp. 1171–1173. [Online]. Available: https://doi.org/10.1109/ASE51524.2021.9678841

[13] M. Sarkar, X. Yan, S. Nateghi, B. J. Holmes, K. G. Vamvoudakis, and A. Homaifar, "A framework for testing and evaluation of operational performance of multi-uav systems," in *Intelligent Systems and Applications - Proceedings of the 2021 Intelligent Systems Conference, IntelliSys 2021, Amsterdam, The Netherlands, 2-3 September, 2021, Volume 1*, ser. Lecture Notes in Networks and Systems, K. Arai, Ed., vol. 294. Springer, 2021, pp. 355–374. [Online]. Available: https://doi.org/10.1007/978-3-030-82193-7_24

[14] R. Delgado, M. Campusano, and A. Bergel, "Fuzz testing in behavior-based robotics," in *IEEE International Conference on Robotics and Automation, ICRA 2021, Xi'an, China, May 30 - June 5, 2021*. IEEE, 2021, pp. 9375–9381. [Online]. Available: https://doi.org/10.1109/ICRA48506.2021.9561259

[15] A. Afzal, D. S. Katz, C. Le Goues, and C. S. Timperley, "Simulation for robotics test automation: Developer perspectives," in *Conference on Software Testing, Verification and Validation*. IEEE, 2021, pp. 263–274.

[16] C. S. Timperley, A. Afzal, D. S. Katz, J. M. Hernandez, and C. Le Goues, "Crashing simulated planes is cheap: Can simulation detect robotics bugs early?" in *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2018, pp. 331–342.

[17] D. Wang, S. Li, G. Xiao, Y. Liu, and Y. Sui, "An exploratory study of autopilot software bugs in unmanned aerial vehicles," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 20–31.

[18] Z. Huang, Y. Shen, J. Li, M. Fey, and C. Brecher, "A survey on ai-driven digital twins in industry 4.0: Smart manufacturing and advanced robotics," *Sensors*, vol. 21, no. 19, p. 6340, 2021. [Online]. Available: https://doi.org/10.3390/s21196340

[19] K. Bojarczuk, N. Gucevska, S. M. M. Lucas, I. Dvortsova, M. Harman, E. Meijer, S. Sapora, J. George, M. Lomeli, and R. Rojas, "Measurement challenges for cyber cyber digital twins: Experiences from the deployment of facebook's WW simulation system," in *International Symposium on Empirical Software Engineering and Measurement*. ACM, 2021, pp. 2:1–2:10. [Online]. Available: https://doi.org/10.1145/3475716.3484196

[20] A. Piazzoni, J. Cherian, M. Azhar, J. Y. Yap, J. L. W. Shung, and R. Vijay, "Vista: a framework for virtual scenario-based testing of autonomous vehicles," in *International Conference on Artificial Intelligence Testing*. IEEE, 2021, pp. 143–150. [Online]. Available: https://doi.org/10.1109/AITEST52744.2021.00035

[21] C. Birchler, N. Ganz, S. Khatiri, A. Gambi, and S. Panichella, "Cost-effective simulation-based test selection in self-driving cars software with sdc-scissor," in *the 29th IEEE International Conference on Software Analysis, Evolution, and Reengineering*, 2022.

[22] V. Nguyen, S. Huber, and A. Gambi, "SALVO: automated generation of diversified tests for self-driving cars from existing maps," in *International Conference on Artificial Intelligence Testing*. IEEE, 2021, pp. 128–135. [Online]. Available: https://doi.org/10.1109/AITEST52744.2021.00033

[23] C. Birchler, S. Khatiri, P. Derakhshanfar, S. Panichella, and A. Panichella, "Single and multi-objective test cases prioritization for self-driving cars in virtual environments," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2022.

[24] M. Alcon, H. Tabani, J. Abella, and F. J. Cazorla, "Enabling unit testing of already-integrated AI software systems: The case of apollo for autonomous driving," in *Euromicro Conference on Digital System Design*. IEEE, 2021, pp. 426–433. [Online]. Available: https://doi.org/10.1109/DSD53832.2021.00071

[25] F. Wotawa, "On the use of available testing methods for verification & validation of ai-based software and systems," in *Proceedings of the Workshop on Artificial Intelligence Safety 2021 (SafeAI 2021) co-located with the Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI 2021), Virtual, February 8, 2021*, ser. CEUR Workshop Proceedings, H. Espinoza, J. McDermid, X. Huang, M. Castillo-Effen, X. C. Chen, J. Hernández-Orallo, S. Ó. hÉigeartaigh, and R. Mallah, Eds., vol. 2808. CEUR-WS.org, 2021. [Online]. Available: http://ceur-ws.org/Vol-2808/Paper_29.pdf

[26] N. P. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems, Sendai, Japan, September 28 - October 2, 2004*. IEEE, 2004, pp. 2149–2154. [Online]. Available: https://doi.org/10.1109/IROS.2004.1389727

[27] S. Artzi, S. Kim, and M. D. Ernst, "Recrash: Making software failures reproducible by preserving object states," in *ECOOP 2008 - Object-Oriented Programming, 22nd European Conference, Paphos, Cyprus, July 7-11, 2008, Proceedings*, ser. Lecture Notes in Computer Science, J. Vitek, Ed., vol. 5142. Springer, 2008, pp. 542–565. [Online]. Available: https://doi.org/10.1007/978-3-540-70592-5_23

[28] S. Narayanasamy, G. Pokam, and B. Calder, "Bugnet: Continuously recording program execution for deterministic replay debugging," in *32st International Symposium on Computer Architecture (ISCA 2005), 4-8 June 2005, Madison, Wisconsin, USA*. IEEE Computer Society, 2005, pp. 284–295. [Online]. Available: https://doi.org/10.1109/ISCA.2005.16

[29] M. Soltani, A. Panichella, and A. van Deursen, "Search-based crash reproduction and its impact on debugging," *IEEE Trans. Software Eng.*, vol. 46, no. 12, pp. 1294–1317, 2020. [Online]. Available: https://doi.org/10.1109/TSE.2018.2877664

[30] W. Jin and A. Orso, "Bugredux: Reproducing field failures for in-house debugging," in *34th International Conference on Software Engineering, ICSE 2012, June 2-9, 2012, Zurich, Switzerland*, M. Glinz, G. C. Murphy, and M. Pezzè, Eds. IEEE Computer Society, 2012, pp. 474–484. [Online]. Available: https://doi.org/10.1109/ICSE.2012.6227168

[31] P. Derakhshanfar, X. Devroey, A. Panichella, A. Zaidman, and A. van Deursen, "Botsing, a search-based crash reproduction framework for java," in *International Conference on Automated Software Engineering*. IEEE, 2020, pp. 1278–1282.

[32] M. Soltani, A. Panichella, and A. van Deursen, "A guided genetic algorithm for automated crash reproduction," in *International Conference on Software Engineering*. IEEE / ACM, 2017, pp. 209–220. [Online]. Available: https://doi.org/10.1109/ICSE.2017.27

[33] A. Gambi, T. Huynh, and G. Fraser, "Generating effective test cases for self-driving cars from police reports," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, M. Dumas, D. Pfahl, S. Apel, and A. Russo, Eds. ACM, 2019, pp. 257–267. [Online]. Available: https://doi.org/10.1145/3338906.3338942

[34] anonymous, "replication package of the paper "Simulation-based Test Case Generation for Unmanned Aerial Vehicles in the Neighborhood of Real Flight"," 2022. [Online]. Available: https://doi.org/10.5281/zenodo.6525021

[35] S. D. Pendleton, H. Andersen, X. Du, X. Shen, M. Meghjani, Y. H. Eng, D. Rus, and M. H. Ang, "Perception, planning, control, and coordination for autonomous vehicles," *Machines*, vol. 5, no. 1, 2017. [Online]. Available: https://www.mdpi.com/2075-1702/5/1/6

[36] C. Forster, M. Pizzoli, and D. Scaramuzza, "Svo: Fast semi-direct monocular visual odometry," in *International Conference on Robotics and Automation (ICRA)*, 2014, pp. 15–22.

[37] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018. [Online]. Available: https://doi.org/10.1109/TRO.2018.2853729

[38] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. I. Nieto, "Voxblox: Incremental 3d euclidean signed distance fields for on-board MAV planning," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2017, Vancouver, BC, Canada, September 24-28, 2017*. IEEE, 2017, pp. 1366–1373. [Online]. Available: https://doi.org/10.1109/IROS.2017.8202315

[39] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.

[40] C. Richter, A. Bry, and N. Roy, "Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments," in *Robotics Research - International Symposium ISRR*, ser. Springer Tracts in Advanced Robotics, vol. 114. Springer, 2013, pp. 649–666. [Online]. Available: https://doi.org/10.1007/978-3-319-28872-7_37

[41] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *Sci. Robotics*, vol. 6, no. 56, p. 1221, 2021. [Online]. Available: https://doi.org/10.1126/scirobotics.abh1221

[42] C. C. D. W. UK, "Accidents will happen - a review of military drone crash data as the uk considers allowing large military drone flights in its airspace," 2019. [Online]. Available: https://dronewars.net/wp-content/uploads/2019/06/DW-Accidents-WEB.pdf

[43] Y. Chen and N. O. Pérez-Arancibia, "Controller synthesis and performance optimization for aerobatic quadrotor flight," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2204–2219, 2020.

[44] PX4, "Px4 simulation," https://docs.px4.io/v1.12/en/simulation/, 2021.

[45] PX4.io, "Log analysis using flight review | px4 user guide," 2022, accessed: 07.02.2022. [Online]. Available: https://docs.px4.io/master/en/log/flight_review.html

[46] ——, "Px4 autopilot flight review," 2022, accessed: 07.02.2022. [Online]. Available: https://logs.px4.io/browse

[47] ——, "Obstacle avoidance | px4 user guide," 2022, accessed: 07.02.2022. [Online]. Available: https://docs.px4.io/master/en/computer_vision/obstacle_avoidance.html

[48] D. J. Berndt and J. Clifford, "Using dynamic time warping to find patterns in time series." in *KDD workshop*, vol. 10, no. 16. Seattle, WA, USA:, 1994, pp. 359–370.

[49] A. Afzal, "Automated testing of robotic and cyberphysical systems," Ph.D. dissertation, Carnegie Mellon University, 2021.

[50] F. Petitjean, A. Ketterlin, and P. Gançarski, "A global averaging method for dynamic time warping, with applications to clustering," *Pattern recognition*, vol. 44, no. 3, pp. 678–693, 2011.

[51] PX4.io, "Gazebo simulation | px4 user guide," 2022, accessed: 07.02.2022. [Online]. Available: https://docs.px4.io/master/en/simulation/gazebo.html

[52] T. Eiter and H. Mannila, "Computing discrete fréchet distance," 1994.

[53] A. Vargha and H. D. Delaney, "A critique and improvement of the cl common language effect size statistics of mcgraw and wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.

[54] M. Lindvall, A. Porter, G. Magnusson, and C. Schulze, "Metamorphic model-based testing of autonomous systems," in *2017 IEEE/ACM 2nd International Workshop on Metamorphic Testing (MET)*. IEEE, 2017, pp. 35–41.

[55] T. Woodlief, S. Elbaum, and K. Sullivan, "Fuzzing mobile robot environments for fast automated crash detection," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 5417–5423.

[56] C. Hildebrandt and S. Elbaum, "World-in-the-loop simulation for autonomous systems validation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 10 912–10 919.

[57] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu, J. Zhao, and Y. Wang, "Deepgauge: Multi-granularity testing criteria for deep learning systems," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, ser. ASE 2018. ACM, 2018, pp. 120–131. [Online]. Available: http://doi.acm.org/10.1145/3238147.3238202

[58] X. Xie, L. Ma, F. Juefei-Xu, M. Xue, H. Chen, Y. Liu, J. Zhao, B. Li, J. Yin, and S. See, "Deephunter: A coverage-guided fuzz testing framework for deep neural networks," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2019. Association for Computing Machinery, 2019, pp. 146–157. [Online]. Available: https://doi.org/10.1145/3293882.3330579

[59] S. Demir, H. F. Eniser, and A. Sen, "Deepsmartfuzzer: Reward guided test generation for deep learning," in *Workshop on Artificial Intelligence Safety 2020 (IJCAI-PRICAI 2020)*, ser. CEUR Workshop Proceedings. CEUR-WS.org, pp. 134–140.

[60] S. Dola, M. B. Dwyer, and M. L. Soffa, "Distribution-aware testing of neural networks using generative models," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 226–237.

[61] L. Ma, F. Juefei-Xu, M. Xue, B. Li, L. Li, Y. Liu, and J. Zhao, "DeepCT: Tomographic combinatorial testing for deep learning systems," in *26th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2019, Hangzhou, China, February 24-27, 2019*. IEEE, pp. 614–618.

[62] A. Stocco, M. Weiss, M. Calzana, and P. Tonella, "Misbehaviour prediction for autonomous driving systems," in *International Conference on Software Engineering*, 2020, pp. 359–371.

[63] G. Jahangirova, A. Stocco, and P. Tonella, "Quality metrics and oracles for autonomous vehicles testing," in *14th IEEE Conference on Software Testing, Verification and Validation, ICST 2021, Porto de Galinhas, Brazil, April 12-16, 2021*. IEEE, 2021, pp. 194–204. [Online]. Available: https://doi.org/10.1109/ICST49551.2021.00030

[64] K. Pei, Y. Cao, J. Yang, and S. Jana, "Deepxplore: Automated whitebox testing of deep learning systems," *Commun. ACM*, no. 11, p. 137?145, Oct.

[65] J. Kim, R. Feldt, and S. Yoo, "Guiding deep learning system testing using surprise adequacy," in *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019*. IEEE / ACM, 2019, pp. 1039–1049. [Online]. Available: https://doi.org/10.1109/ICSE.2019.00108

[66] A. Gambi, M. Müller, and G. Fraser, "Automatically testing self-driving cars with search-based procedural content generation," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, Beijing, China, July 15-19, 2019*. ACM, pp. 318–328.

[67] V. Riccio and P. Tonella, "Model-based exploration of the frontier of behaviours for deep learning system testing," in *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, 2020, pp. 876–888.

[68] R. B. Abdessalem, S. Nejati, L. C. Briand, and T. Stifter, "Testing vision-based control systems using learnable evolutionary algorithms," in *International Conference on Software Engineering*. ACM, 2018, pp. 1016–1026. [Online]. Available: http://doi.acm.org/10.1145/3180155.3180160